

Model evaluation

Module containing evaluation functions suitable for judging the performance of a fitted LightFM model.

lightfm.evaluation.auc_score(*model*, *test_interactions*, *train_interactions=None*, *user_features=None*, *item_features=None*, *preserve_rows=False*, *num_threads=1*, *check_intersections=True*) [\[source\]](#)

Measure the ROC AUC metric for a model: the probability that a randomly chosen positive example has a higher score than a randomly chosen negative example. A perfect score is 1.0.

Parameters:

- **model** (*LightFM instance*) – the fitted model to be evaluated
- **test_interactions** (*np.float32 csr_matrix of shape [n_users, n_items]*) – Non-zero entries representing known positives in the evaluation set.
- **train_interactions** (*np.float32 csr_matrix of shape [n_users, n_items], optional*) – Non-zero entries representing known positives in the train set. These will be omitted from the score calculations to avoid re-recommending known positives.
- **user_features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features.
- **item_features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features.
- **preserve_rows** (*boolean, optional*) – When False (default), the number of rows in the output will be equal to the number of users with interactions in the evaluation set. When True, the number of rows in the output will be equal to the number of users.
- **num_threads** (*int, optional*) – Number of parallel computation threads to use. Should not be higher than the number of physical cores.
- **check_intersections** (*bool, optional, True by default,*) – Only relevant when train_interactions are supplied. A flag that signals whether the test and train matrices should be

checked for intersections to prevent optimistic ranks / wrong evaluation / bad data split.

Returns: Numpy array containing AUC scores for each user. If there are no interactions for a given user the returned AUC will be 0.5.

Return type: np.array of shape [n_users with interactions or n_users,]

lightfm.evaluation.precision_at_k(*model, test_interactions, train_interactions=None, k=10, user_features=None, item_features=None, preserve_rows=False, num_threads=1, check_intersections=True*) [\[source\]](#)

Measure the precision at k metric for a model: the fraction of known positives in the first k positions of the ranked list of results. A perfect score is 1.0.

Parameters:

- **model** (*LightFM instance*) – the fitted model to be evaluated
- **test_interactions** (*np.float32 csr_matrix of shape [n_users, n_items]*) – Non-zero entries representing known positives in the evaluation set.
- **train_interactions** (*np.float32 csr_matrix of shape [n_users, n_items], optional*) – Non-zero entries representing known positives in the train set. These will be omitted from the score calculations to avoid re-recommending known positives.
- **k** (*integer, optional*) – The k parameter.
- **user_features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features.
- **item_features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features.
- **preserve_rows** (*boolean, optional*) – When False (default), the number of rows in the output will be equal to the number of users with interactions in the evaluation set. When True, the number of rows in the output will be equal to the number of users.
- **num_threads** (*int, optional*) – Number of parallel computation threads to use. Should not be higher than the number of physical cores.
- **check_intersections** (*bool, optional, True by default,*) – Only relevant when train_interactions are supplied. A flag that signals whether the test and train matrices should be checked for intersections to prevent optimistic ranks / wrong evaluation / bad data split.

Returns: Numpy array containing precision@k scores for each user. If there are no interactions for a given user the returned precision will be 0.

Return type: np.array of shape [n_users with interactions or n_users,]

lightfm.evaluation.recall_at_k(*model, test_interactions, train_interactions=None, k=10, user_features=None, item_features=None, preserve_rows=False, num_threads=1, check_intersections=True*) [\[source\]](#)

Measure the recall at k metric for a model: the number of positive items in the first k positions of the ranked list of results divided by the number of positive items in the test period. A perfect score is 1.0.

Parameters:

- **model** (*LightFM instance*) – the fitted model to be evaluated
- **test_interactions** (*np.float32 csr_matrix of shape [n_users, n_items]*) – Non-zero entries representing known positives in the evaluation set.
- **train_interactions** (*np.float32 csr_matrix of shape [n_users, n_items], optional*) – Non-zero entries representing known positives in the train set. These will be omitted from the score calculations to avoid re-recommending known positives.
- **k** (*integer, optional*) – The k parameter.
- **user_features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features.
- **item_features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features.
- **preserve_rows** (*boolean, optional*) – When False (default), the number of rows in the output will be equal to the number of users with interactions in the evaluation set. When True, the number of rows in the output will be equal to the number of users.
- **num_threads** (*int, optional*) – Number of parallel computation threads to use. Should not be higher than the number of physical cores.
- **check_intersections** (*bool, optional, True by default,*) – Only relevant when train_interactions are supplied. A flag that signals whether the test and train matrices should be checked for intersections to prevent optimistic ranks / wrong evaluation / bad data split.

Returns: Numpy array containing recall@k scores for each user. If there are no interactions for a given user having items in the test period, the returned recall will be 0.

Return type: np.array of shape [n_users with interactions or n_users,]

lightfm.evaluation.reciprocal_rank(*model*, *test_interactions*, *train_interactions=None*, *user_features=None*, *item_features=None*, *preserve_rows=False*, *num_threads=1*, *check_intersections=True*) [\[source\]](#)

Measure the reciprocal rank metric for a model: $1 / \text{the rank of the highest ranked positive example}$. A perfect score is 1.0.

Parameters:

- **model** (*LightFM instance*) – the fitted model to be evaluated
- **test_interactions** (*np.float32 csr_matrix of shape [n_users, n_items]*) – Non-zero entries representing known positives in the evaluation set.
- **train_interactions** (*np.float32 csr_matrix of shape [n_users, n_items], optional*) – Non-zero entries representing known positives in the train set. These will be omitted from the score calculations to avoid re-recommending known positives.
- **user_features** (*np.float32 csr_matrix of shape [n_users, n_user_features], optional*) – Each row contains that user's weights over features.
- **item_features** (*np.float32 csr_matrix of shape [n_items, n_item_features], optional*) – Each row contains that item's weights over features.
- **preserve_rows** (*boolean, optional*) – When False (default), the number of rows in the output will be equal to the number of users with interactions in the evaluation set. When True, the number of rows in the output will be equal to the number of users.
- **num_threads** (*int, optional*) – Number of parallel computation threads to use. Should not be higher than the number of physical cores.
- **check_intersections** (*bool, optional, True by default,*) – Only relevant when train_interactions are supplied. A flag that signals whether the test and train matrices should be checked for intersections to prevent optimistic ranks / wrong evaluation / bad data split.

Returns: Numpy array containing reciprocal rank scores for each user. If there are no interactions for a given user the returned value will be 0.0

be 0.0.

Return type: np.array of shape [n_users with interactions or n_users,]